UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

CS2042 - BASES DE DATOS II

> Reinventa el mundo <

Organización Física y Almacenamiento

SEMANA 02

Heider Sanchez
hsanchez@utec.edu.pe

TRANSFORMATEC

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

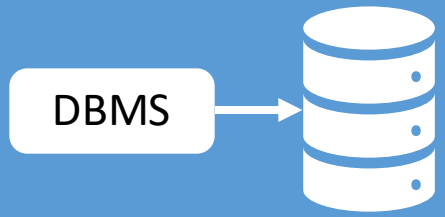
1.

Organización de Archivos

Conceptos y Fundamentos

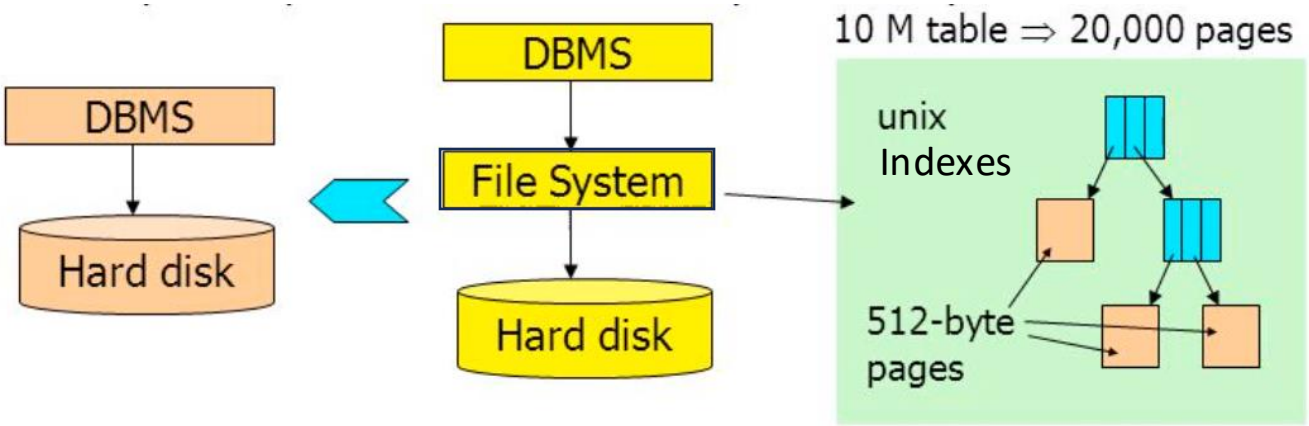
File Structures: An object-oriented approach with C++, Michael J. Folk. Addison Wesley, 3rd Edition, 1998.

Conceptos



- ¿Cómo se almacenan realmente los datos en una base de datos?
- ¿Por qué algunas consultas son más rápidas que otras?
- ¿Cómo impacta la organización de archivos en el rendimiento del análisis de datos y machine learning?

Conceptos



Tipos de Organización de Archivos

1. **Archivos Secuenciales** – Útiles para lecturas masivas (logs, ETL).
2. **Archivos Indexados** – Permite búsquedas rápidas.

Conceptos

Una base de datos es almacenado como una **colección de archivos**.

Cada archivo se organiza en páginas

Cada página contiene una secuencia de **registros**.

Un registro es una secuencia de **campos**.

- un archivo ⇔ una tabla
- un registro ⇔ una tupla
- un campo ⇔ una columna

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Conceptos

Key: Es un atributo que identifica de **forma única** cada registro, como la clave primaria en una base de datos. Permite **acceder rápidamente** a la información y **evitar duplicados**.

Page: Cuando un archivo es demasiado grande, **se divide en páginas** (bloques de igual tamaño) para facilitar su manejo en memoria. Estas páginas son la unidad **de intercambio entre el disco y la memoria** principal, permitiendo operaciones como inserción, modificación, eliminación y búsqueda.

Index: Es un puntero a un registro en un archivo, que permite acceder a los datos de forma más rápida y eficiente, **evitando búsquedas secuenciales.**

Conceptos

Los archivos pueden ser:

- **Archivos de texto**

  Guardan datos como caracteres (ej. números como “123”).
- **Archivos binarios**

  Almacenan datos en formato binario (ej. números en código máquina).

Operaciones básicas en archivos

Read: Recupera registros de un archivo. Puede ser **secuencial** (uno tras otro) o **directa** (usando un índice o clave). La eficiencia depende de la estructura del archivo y el uso de buffers.

Write: Agrega registros al archivo. Puede hacerse al **final**, en una **posición específica**, o **sobrescribiendo** registros existentes. La reorganización y fragmentación pueden afectar el rendimiento.

Delete: Remueve registros mediante **eliminación lógica** (marcado como inactivo) o **física** (borrado definitivo). Puede requerir reorganización o reutilización del espacio.

Operaciones básicas en archivos con C++

Class	Contents
filebuf	Its purpose is to set the file buffers to read and write. Contains **Openprot** constant used in the **open()** of file stream classes.Also contain **close()** and **open()** as members.
fstreambase	Provides operations common to file streams.Serves as a base for **fstream,ifstream** and **ofstream** class.Contains **open()** and **close()** functions.
ifstream	Provides input operations.Contains **open()** with default input mode.Inherits the functions **get(),getline(),read(),seekg(),tellg()** functions from istream.
ofstream	Provides output operations.Contains **open()** with default output mode.Inherits **put(),seekp(),tellp()** and **write()** functions from **ostream**.
fstream	Provides support for simultaneous input and output operations.Contains open with default input mode.Inherits all the functions from **istream** and **ostream** classes through **iostream**.

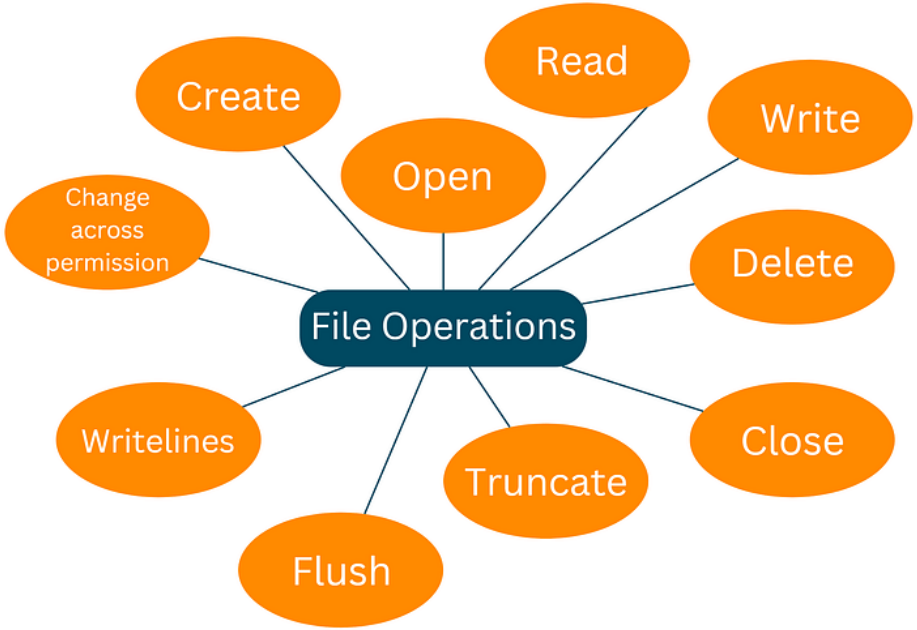
https://www.geeksforgeeks.org/file-handling-c-classes/

Operaciones básicas en archivos con C++

```cpp
#include <iostream>
#include <fstream>
using namespace std;

struct Datos {
    int numero;
    float decimal;
};

void escribir(const string& archivo, const Datos& datos) {
    ofstream ptrFile(archivo, ios::binary);
    if (ptrFile) {
        ptrFile.write(reinterpret_cast<const char*>(&datos),
                      sizeof(Datos));
        ptrFile.close();
        cout << "Datos guardados en " << archivo;
    } else {
        cerr << "No se pudo abrir el archivo.";
    }
}

int main() {
    Datos datos = {42, 3.14f};
    string archivo = "datos.bin";

    escribir(archivo, datos);
    return 0;
}
```

```cpp
#include <iostream>
#include <fstream>
using namespace std;

struct Datos {
    int numero;
    float decimal;
};

Datos leer(const string& archivo) {
    Datos datos;
    ifstream ptrFile(archivo, ios::binary);
    if (ptrFile) {
        ptrFile.read(reinterpret_cast<char*>(&datos),
                     sizeof(Datos));
        ptrFile.close();
        cout << "Número leído: " << datos.numero;
        cout << "Decimal leído: " << datos.decimal;
    } else {
        cerr << "No se pudo abrir el archivo.";
    }
    return datos;
}

int main() {
    string archivo = "datos.bin";
    leer(archivo);
    return 0;
}
```

Operaciones básicas en archivos con Python

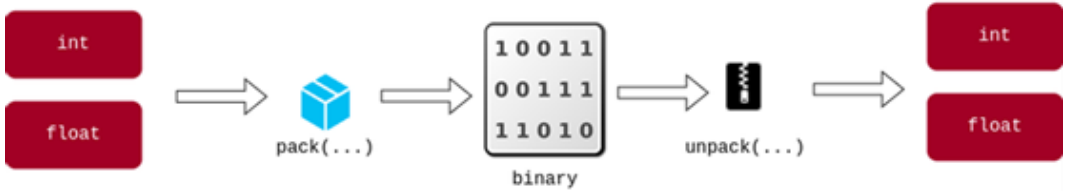



```
import struct
import pikle
import csv
import json
import xml.etree
...
```

[File Handling in Python [Complete Series]]

Operaciones básicas en archivos con Python

import struct



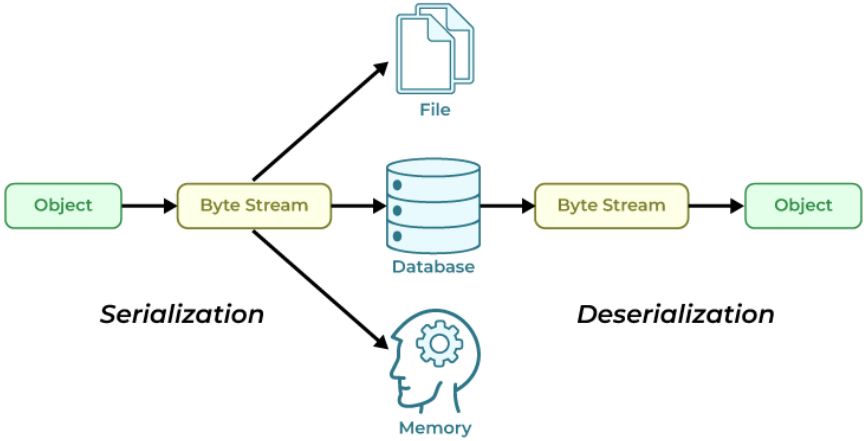

```python
import struct

def escribir(archivo, numero, decimal):
    datos_empaquetados = struct.pack('if', numero, decimal)
    with open(archivo, 'wb') as ptrFile:
        ptrFile.write(datos_empaquetados)
    print(f"Datos guardados en {archivo}")

# Ejemplo de escritura de datos
archivo = 'datos.bin'
escribir(archivo, 42, 3.14)
```

```python
import struct

def leer(archivo):
    with open(archivo, 'rb') as ptrFile:
        contenido = ptrFile.read()
    numero, decimal = struct.unpack('if', contenido)
    print(f"Número leído: {numero}")
    print(f"Decimal leído: {decimal}")
    return numero, decimal

# Ejemplo de lectura de datos
archivo = 'datos.bin'
leer(archivo)
```

https://docs.python.org/3/library/struct.html

Operaciones básicas en archivos con Python

import pickle




```python
import pickle

datos = {
    'nombre': 'Ana',
    'edad': 28,
    'intereses': ['música', 'viajes', 'lectura']
}

with open('datos.pkl', 'wb') as archivo:
    pickle.dump(datos, archivo)

print("Datos guardados exitosamente.")
```

```python
import pickle

# Leer desde archivo binario
with open('datos.pkl', 'rb') as archivo:
    datos_recuperados = pickle.load(archivo)

print("Datos leídos:")
print(datos_recuperados)
```

https://www.geeksforgeeks.org/understanding-python-pickling-example/

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

> Reinventa el mundo <

2.

Modelos de Almacenamiento

1- Registros de Longitud Fija
2- Registros de Longitud Variable

File Structures: An object-oriented approach with C++, Michael J. Folk. Addison Wesley, 3rd Edition, 1998.

Heap Files

Un heap file es la estructura de almacenamiento más básica: un conjunto de páginas donde los registros se insertan sin orden.

Ventajas

- Inserciones rápidas (append al final)
- Estructura simple, sin mantenimiento de orden
- Ideal para cargas de trabajo INSERT-heavy
- Sin overhead de índice en escritura

Desventajas

- Búsqueda secuencial: O(n) páginas
- Sin garantía de orden en recuperación
- Espacio fragmentado con DELETEs frecuentes
- Requiere VACUUM para recuperar espacio

Registros de Longitud Fija

- Todos los registros en un archivo tienen **la misma longitud** y **cantidad de campos**

Tipo de dato	Descripción	Tamaño fijo
`INTEGER` / `INT`	Entero de 4 bytes	4 bytes
`SMALLINT`	Entero pequeño	2 bytes
`BIGINT`	Entero grande	8 bytes
`REAL`	Número de punto flotante	4 bytes
`DOUBLE PRECISION`	Número de punto flotante doble	8 bytes
`BOOLEAN`	Verdadero o falso	1 byte
`CHAR(n)`	Cadena de longitud fija	n bytes
`DATE`	Fecha (sin hora)	4 bytes
`TIME`	Hora (sin fecha)	8 bytes

Tipo de datos de longitud fija en PostgreSQL

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Registros de Longitud Fija

- Todos los registros en un archivo tienen **la misma longitud** y **cantidad de campos**.
- Cada campo tiene un tamaño fijo, lo que permite ubicar los valores fácilmente, ya que sus **posiciones están predeterminadas**.

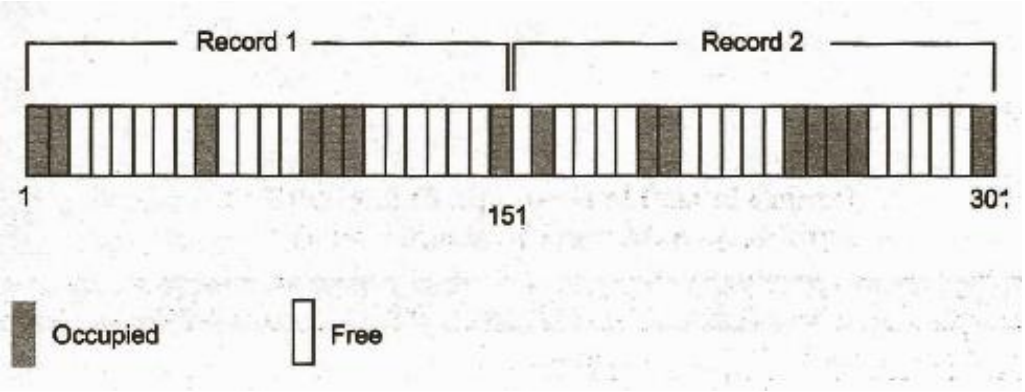
archivo.dat

0	Howard	Paredes	Zegarra	Computacion
51	Penny	Vargas	Cordero	Industrial
	12	12	12	15

Offset = seekg(i * 51)

Registros de Longitud Fija: Acceso Directo

- Como todos los registros tienen el mismo tamaño, es fácil calcular su **posición exacta** y acceder directamente a ellos sin necesidad de recorrer todo el archivo.




```
Alumno record;
size = sizeof(record);
offset = i * size;
```

i: posición lógica del registro
offset: posición física del registro

Registros de Longitud Fija: Ejemplo C++

Record

```
class Alumno
{
public:
  char Nombre [12];
  char Apellidos [12];
  int edad;
};
```

Archivo de Texto

```
void escribirRegistro(const char* filename,
                      const Alumno &record) {
    ofstream file(filename, ios::app);
    file << left << setw(12) << record.Nombre
         << left << setw(12) << record.Apellidos
         << setw(4) << record.edad
         << "\n";
    file.close();
}

Alumno leerRegistro(const char* filename,
                    int pos) {
    ifstream file(filename);
    Alumno record;
    file.seekg(pos * (12 + 12 + 4 + 1), ios::beg);
    file.read(record.Nombre, 12);
    file.read(record.Apellidos, 12);
    file >> record.edad;
    file.close();
    return record;
}
```

Registros de Longitud Fija: Ejemplo C++

Record

```
class Alumno
{
public:
  char Nombre [12];
  char Apellidos [12];
  int edad;
};
```

Archivo Binario

```
void escribirRegistro (const char* filename,
                       const Alumno &record)
{
    ofstream file(filename, ios::binary | ios::app);
    file.write((const char*)(&record), sizeof(Alumno));
    file.close();
}

Alumno leerRegistro (const char* filename,
                     int pos)
{
    ifstream file(filename, ios::binary);
    Alumno record;
    file.seekg(pos * sizeof(Alumno), ios::beg);
    file.read((char*)(&record), sizeof(Alumno));
    file.close();
    return record;
}
```

Registros de Longitud Fija: Ejemplo Python

Archivo de Texto

```
def escribirRegistro (filename, alumno):
    with open(filename, "a") as file:  # Modo append
        # 12+12+3=27 caracteres fijos
        file.write(f"{alumno.nombre:<12}{alumno.apellido:<12}{alumno.edad:03}\n")

def leerRegistro (filename, pos):
    with open(filename, "r") as file:
        file.seek(pos * 27)  # Mover el puntero a la posición indicada
        line = file.read(27)
        if line:
            nombre = line[:12].strip()
            apellido = line[12:24].strip()
            edad = int(line[24:27].strip())
            return Alumno(nombre, apellido, edad)
        return None
```

Registros de Longitud Fija: Ejemplo Python

Archivo Binario

```
import struct

FORMAT = '12s12si'  # 12 bytes para nombre, 12 para apellido, 4 para edad (int)
RECORD_SIZE = struct.calcsize(FORMAT)  # 28 bytes por registro

def escribirRegistro (filename, alumno):
    with open(filename, "ab") as file:  # Modo append binary
        record = struct.pack(FORMAT, alumno.nombre.encode(),
                             alumno.apellido.encode(), alumno.edad)
        file.write(record)

def leerRegistro (filename, pos):
    with open(filename, "rb") as file:
        file.seek(pos * RECORD_SIZE)  # Posicionamiento directo
        data = file.read(RECORD_SIZE)
        if not data:
            return None
        nombre, apellido, edad = struct.unpack(FORMAT, data)
        return Alumno(nombre.decode().strip(), apellido.decode().strip(), edad)
```

Registros de Longitud Fija

- **Problemas**:
  - El acceso a los registros es simple pero se corre el riesgo de cruzar bloques.
  - La modificación no permite que los registros crucen el límite del bloque.
  - ¿Eliminación de registros?

	Id	Nombre	Ciclo
1	P-102	Andrea	5
2	P-251	Carlos	7
3	P-412	Maria	3
4	P-255	Juan	7
5	P-250	Javier	7
6	P-312	Mabel	3
7	P-982	Saulo	9

Registros de Longitud Fija: Eliminación

- **Alternativa 1**:
  - Mover los registros *i+1,...,n* hacia *i,...,n-1*

¿En donde mantener el size?

¿Complejidad?

	Id	Nombre	Ciclo
1	P-102	Andrea	5
2	P-251	Carlos	7
3	P-412	Maria	3
4	P-255	Juan	7
5	P-250	Javier	7
6	P-312	Mabel	3
7	P-982	Saulo	9

Registros de Longitud Fija: Eliminación

- **Alternativa 1**:
  - Mover los registros *i+1,...,n* hacia *i,...,n-1*

¿En donde mantener el size?

¿Complejidad?

	Id	Nombre	Ciclo
1	P-102	Andrea	5
2	P-412	Maria	3
3	P-255	Juan	7
4	P-250	Javier	7
5	P-312	Mabel	3
6	P-982	Saulo	9
7	P-982	Saulo	9

Registros de Longitud Fija: Eliminación

- **Alternativa 2**:
  - Mover el registro *n* hacia *i*

¿Complejidad?

	Id	Nombre	Ciclo
1	P-102	Andrea	5
2	P-251	Carlos	7
3	P-412	Maria	3
4	P-255	Juan	7
5	P-250	Javier	7
6	P-312	Mabel	3
7	P-982	Saulo	9

Registros de Longitud Fija: Eliminación

- **Alternativa 2**:
  - Mover el registro *n* hacia *i*

Size = 5

	Id	Nombre	Ciclo
1	P-102	Andrea	5
2	P-982	Saulo	9
3	P-312	Mabel	3
4	P-255	Juan	7
5	P-250	Javier	7
6	P-312	Mabel	3
7	P-982	Saulo	9

Registros de Longitud Fija: Eliminacion

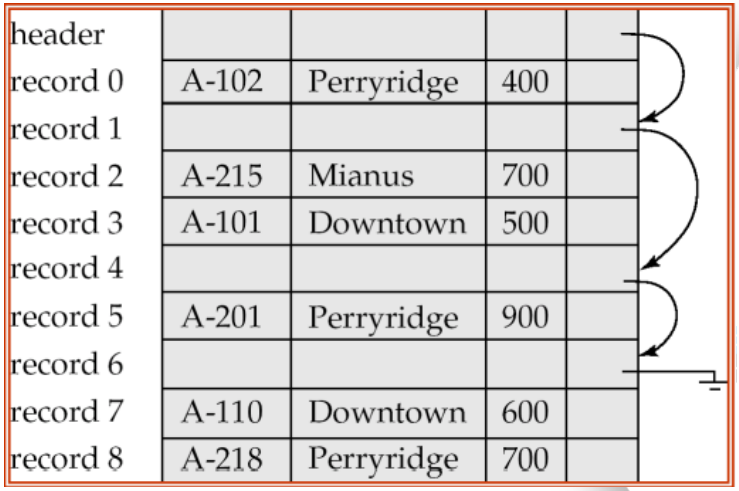
- **Alternativa 3**:
  - No mover registros, pero enlazar todos los registros liberados en una lista (Free List).

header				
record 0	A-102	Perryridge	400	
record 1				
record 2	A-215	Mianus	700	
record 3	A-101	Downtown	500	
record 4				
record 5	A-201	Perryridge	900	
record 6				
record 7	A-110	Downtown	600	
record 8	A-218	Perryridge	700	

Free List: eliminar registros 6,4 y 1.

Registros de Longitud Fija

- **Free List**:Gestiona espacios de registros eliminados para su reutilización.
- **Cómo funciona**:
  1. El **header** almacena la dirección del primer registro eliminado.
  2. Cada registro eliminado guarda la dirección del siguiente.
  3. Se forma una **lista enlazada** de espacios reutilizables.
- **Optimización**: Usar los mismos registros eliminados para almacenar punteros.



Free List: eliminar registros 6,4 y 1.

- **Free List:**

datos.dat

-1

	Id	Nombre	Ciclo	NextDel
1	P-256	Federico	1	0
2	P-251	Carlos	7	0
3	P-412	Maria	3	0
4	P-255	Juan	7	0
5	P-250	Javier	7	0
6	P-312	Mabel	3	0
7	P-982	Saulo	9	0

Eliminar 3, 5 y 1

	Eliminación	Inserción
FIFO	O( )	O( )
LIFO	O( )	O( )

* k es total de eliminados

Registros de Longitud Variable

- Los registros tienen campos cuyo espacio se ajusta al contenido almacenado. Incluyen una cabecera adicional para indicar la longitud.

Tipo de dato	Descripción	Tamaño variable
`VARCHAR(n)`	Cadena de longitud variable	Hasta n bytes + 4 bytes de cabecera
`TEXT`	Cadena sin límite de longitud	Según contenido
`BYTEA`	Datos binarios	Según contenido
`JSON` / `JSONB`	Datos en formato JSON	Según contenido
`ARRAY`	Arreglo de cualquier tipo	Según contenido
`NUMERIC` / `DECIMAL`	Precisión arbitraria	Según precisión

Tipo de datos de longitud variable en PostgreSQL

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Registros de Longitud Variable

El manejo de archivos con registros de longitud variable es una solución en los sistemas de bases de datos para soportar campos de tamaño dinámico, como **TEXT, JSON y BYTEA**

Ventaja: Permite un uso más eficiente de la memoria, tanto en **RAM** como en **almacenamiento secundario**.

Para identificar el inicio y el fin de cada campo o registro, se emplean métodos específicos:

Delimitadores: Caracteres especiales que separan los campos.	**Indicadores de longitud**: Valores numéricos que indican el tamaño de cada campo o registro.

Registros de Longitud Variable

1. Archivos de Texto: Usa caracteres especiales para separar los campos.

- Los separadores **no deben aparecer** dentro de los valores de los campos.
- Para ubicar un campo, es necesario recorrer el registro hasta encontrarlo.

```
Howard|Paredes|Zegarra|Computacion|5|1500.50
Penny|Vargas|Cordero|Industrial|2|2850.00
```

← \n
← como separador de registro

Problemas:

- El delimitador es parte del contenido.
- Acceso directo a un registro. O(n)
- Eliminar un registro. O(n)

Registros de Longitud Variable

2. Archivos Binarios: Usa **indicadores de longitud** para definir el tamaño de cada campo o registro.

- El indicador de longitud se coloca al **inicio** del campo o registro.
- Solo es necesario especificar el tamaño de **campos de tipo texto**.

```
43:6:Howard7:Paredes7:Zegarra11:Computacion4:58:1500.50
40:5:Penny6:Vargas7:Cordero10:Industrial4:28:2850.00
```

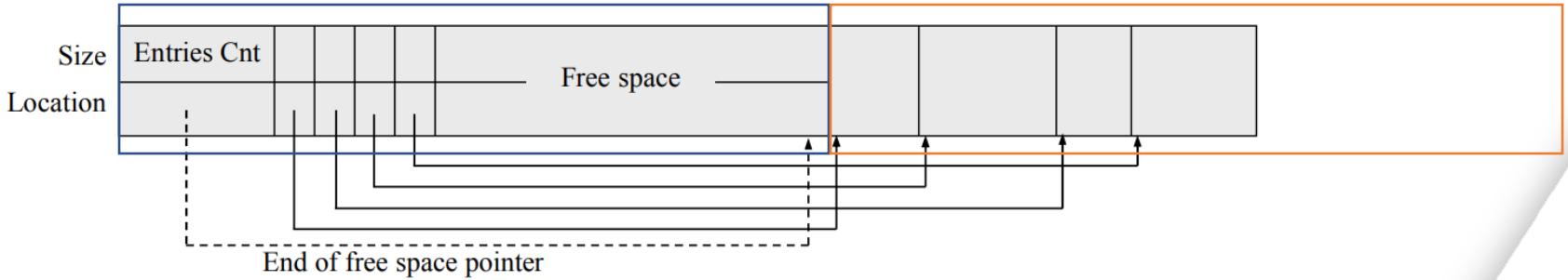
Problemas:

- ~~El delimitador es parte del contenido~~
- Acceso directo a un registro
- Eliminar un registro

https://www.cs.scranton.edu/~mccloske/courses/cmps340/file_record_storage.html

Registros de Longitud Variable

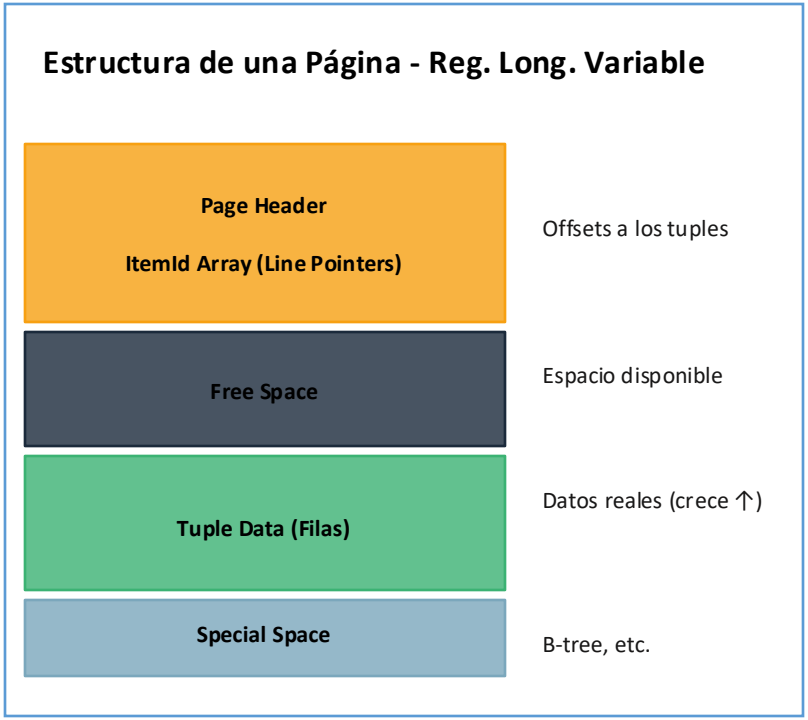
3. Slotted Page: cabecera que indica el inicio de cada registro



- Slotted Page contiene:
- Localización y tamaño de cada registro.
- *El número de registros de entrada.*
  - *El final del espacio libre separado para este encabezado.*
- Para localizar un registro siempre se verifica el encabezado
- Mantener actualizado el encabezado

http://labe.felk.cvut.cz/~stepan/AE3B33OSD/Lesson10-Data_Access.pdf

Registros de Longitud Variable



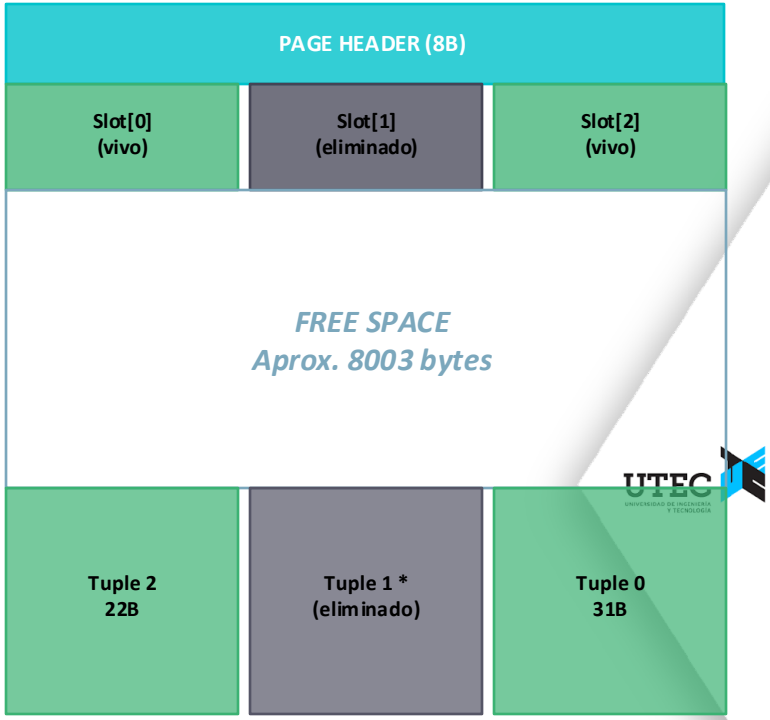
Slotted Page

- Header de página contiene un array de (offset, length)
- Tuplas se insertan desde el final de la página hacia arriba
- Permite compactar sin cambiar el RID externo

Demo: Insertar, Leer y Eliminar en SlottedPage

```
p = SlottedPage()

# Insertar 3 registros (JSON simulado)
s0 = p.insert(b'{id:1,nom:"Ana",dep:"ENG"}')
s1 = p.insert(b'{id:2,nom:"Alejandro García",dep:"MKT"}')
s2 = p.insert(b'{id:3,nom:"Bo",dep:"FIN"}')

print(p._nslots)        # → 3
print(p.free_space())   # → 8 003 bytes libres

# Leer por slot_id — O(1) siempre
print(p.read(0))  # b'{id:1,...}'
print(p.read(1))  # b'{id:2,...}'
print(p.read(2))  # b'{id:3,...}'

# Eliminar slot 1 (dead tuple)
p.delete(1)
print(p.read(1))         # b'' (muerto)

# Slots 0 y 2 siguen válidos e intactos
print(p.read(0))         # ✓ igual
print(p.read(2))         # ✓ igual

# Persistir en disco
open('/tmp/pg0.bin','wb').write(p._buf)
```

Estado de la página



** espacio recuperable con compact()*

slot_id es estable incluso tras deletions — compact() reordena físicamente sin cambiar los slot_ids. Los índices no se invalidan.

Registros de Longitud Variable

Header independiente

Header.dat

	Posición	Tamaño
1	0	18
2	18	21
3	39	20
4	59	24
5	83	24

Datos.txt

		Codigo|Nombre|Apellidos|Carrera
seekg(0) →	1	001|Jose|Lopez|CS
seekg(18) →	2	002|Maria|Vergara|IN
seekg(39) →	3	003|Luis|Vergara|IN
seekg(59) →	4	004|Patricia|Vergara|IN
seekg(83) →	5	005|Valentin|Vergara|IN

Leer el registro i → $T(n) = 1 + 1 = 2$ → $O(1)$

Registros de Longitud Variable

3. Slotted Page: cabecera que indica el inicio de cada registro

Problemas:

- ~~El delimitador es un carácter~~
- ~~Acceso directo a un registro~~
- **¿Eliminar un registro?**

Comparativa: Longitud Fija vs Variable

¿Cuándo elegir cada estrategia? Análisis de trade-offs en el diseño de almacenamiento.

Dimensión	Longitud FIJA	Longitud VARIABLE
Tamaño en disco	Fijo (padding si campo corto)	Real (sin desperdicio)
Acceso a campo i	O(1) — offset = base + i×size	O(1) — leer offset del header
Acceso por slot	O(1) — slot × record_size	O(1) — slot descriptor
Complejidad implementación	Muy simple (memcpy / struct)	Mayor (header + gestión de gaps)
Actualizaciones	Simple si valor mismo tamaño	Puede requerir reubicación
Compactación	No necesita	compact() periódico
Casos de uso	Números, fechas, enums, tipos fijos	Texto libre, JSON, arrays
PostgreSQL usa	int4, float8, date, bool…	VARCHAR, TEXT, JSONB, arrays

PostgreSQL usa un HÍBRIDO: campos fijos al inicio del tuple (alineados, acceso O(1)) y campos variables al final referenciados por tabla de atributos (attbyval/attlen en pg_attribute).

Preguntas

- ¿Qué es el FillFactor en paginas de disco?
- ¿Cuál es el costo de acceso en cada operación?: Scan Secuencial, Búsqueda, Insertar y Eliminar
- ¿Cómo implementar un Heap File en Python/C++ con paginación?

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Conclusiones

Registros de Longitud Fija:

- Son fáciles de manejar y permiten un acceso rápido y directo a cualquier registro, ya que todos tienen el mismo tamaño.
- Pueden desperdiciar espacio si los campos no se utilizan completamente o la necesidad de truncar la información si el dato excede el tamaño fijo.

Registros de Longitud Variable:

- Permiten un uso más eficiente del espacio, ya que solo ocupan el espacio necesario para los datos.
- La gestión de estos registros puede ser más compleja, ya que se necesitan métodos específicos para identificar el inicio y el fin de cada campo o registro.

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

> Reinventa el mundo <

Laboratorio 02

TRANSFORMATEC